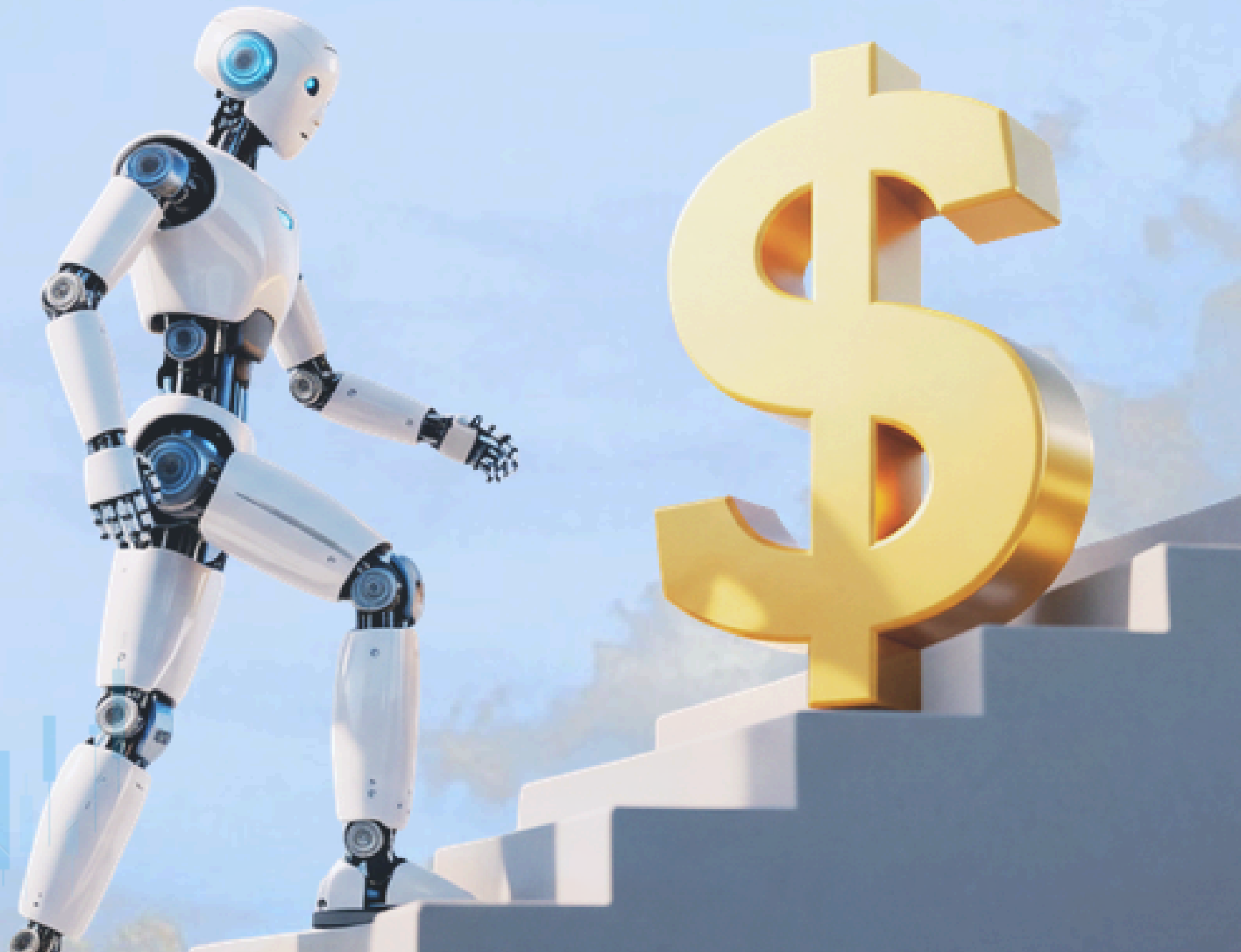


Agent-based framework

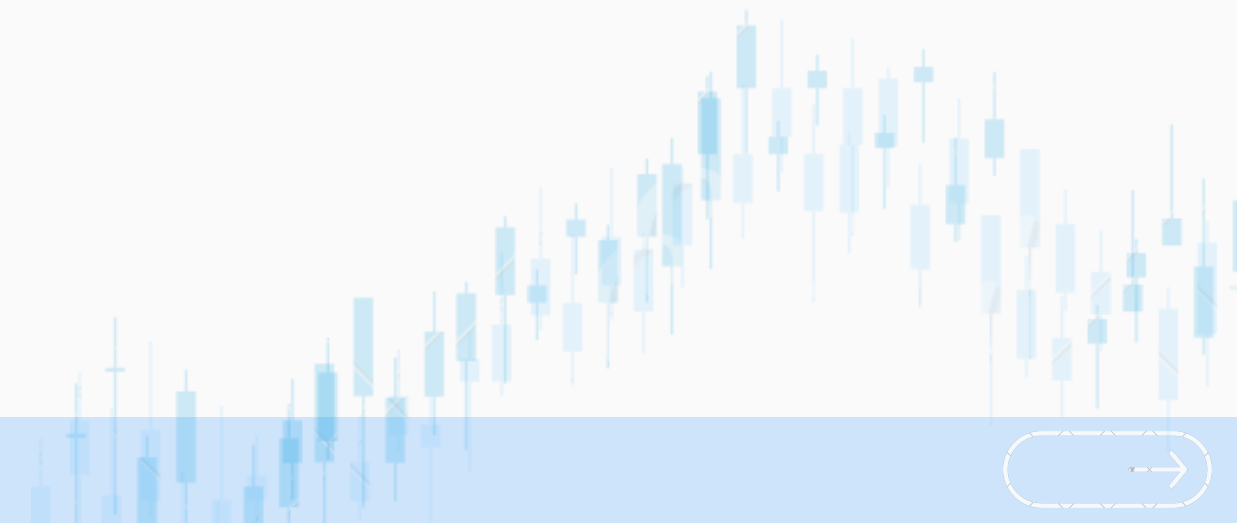
for stock price prediction

Team 2: Aurora, Freddi, Jack, Nadi, Sabina



Agenda

- Motivation & Data
- Approach
- The Pipeline
- Results
- Demo
- Next steps
- Conclusion



Motivation & Data

Motivation:

- Financial markets react to news in real time.
 - Thousands of financial articles appear every day.
 - Manual interpretation is not scalable.
- Need for automated signal extraction from headlines

Can financial news headlines predict whether a stock moves up, down, or stays neutral on the next trading day?

Data Source:

Headlines: FNSPID dataset (HuggingFace)

~15,000 articles across 30 tickers (2009–2023)

Prices: yfinance

closing price on day T and T+1

Label Threshold:

>+1% = up

<-1% = down

±1% = neutral

Filtering: rows with no price match dropped; bottom/top 1% of price moves dropped (~2%) as headlines don't explain extreme moves

Train/test split is time-based:

earliest 80% = train

latest 20% = test

Extra: Karaoglu & Gowda (2026) - arXiv:2606.12210

Jiang & Zeng (2025) - arXiv:2306.02136

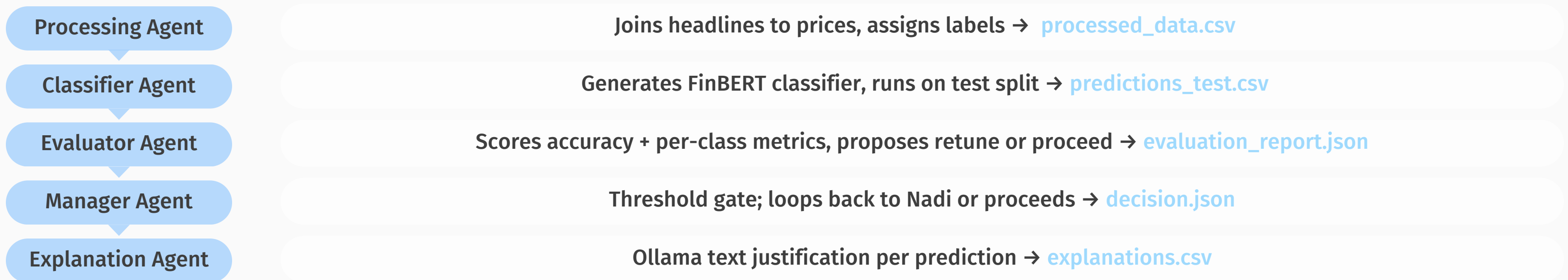
Approach

What we built:

A 5-agent pipeline that reads a financial news headline and predicts whether the stock will move up, down, or neutral the next day, then generates a plain-text explanation for each prediction.

1 agent per team member → passing files to each other in predefined pipeline.

Pipeline:



The Pipeline

Processing Agent

- Joins **FNSPID** news headlines ↔ **yfinance** prices (close on day T and next trading day T+1)
- Computes % change → labels each headline **up** (>+1%), **down** (<-1%), **neutral** (in between)
- Cleans: drops rows with no price data + outliers (1st–99th percentile)
- **Train/test/val split by date**
- **COVID cutoff** (drop post-2019) flag so the crash stays out of the test set
- → processed_data.csv

```
dates = pd.to_datetime(df["date"])
split_cut = dates.quantile(0.8)
val_cut = dates[dates <= split_cut].quantile(0.9) # last 10% of training dates
df["split"] = ["test" if d > split_cut else ("val" if d > val_cut else "train")
               for d in dates]
```

Processing Agent

Classifier Agent

Evaluator Agent

Manager Agent

Explanation Agent

Classifier Agent

- Runs FinBERT to predict up / down / neutral from each headline
- A code-generation agent: writes and runs classifier.py, hands code + predictions on
- Predicts held-out (val + test) rows only - not the training data
- Can load a fine-tuned FinBERT (opt-in) or use pretrained
- Optional guardrailed LLM that rewrites the classifier code - kept only if it passes validation
- → predictions_test.csv

```
def generate_code(state: PipelineState) -> dict:
    """LangGraph node to read parameters from state/retune request and write classifier.py."""
    threshold = 0.5
    max_length = 128
    focus_labels = []
    boost_factor = 1.25

    # Read from retune request if it exists in state
    retune_req = state.get("retune_request")
    if retune_req:
        suggested = retune_req.get("suggested_params", {})
        threshold = suggested.get("threshold", threshold)
        max_length = suggested.get("max_length", max_length)
        boost_factor = suggested.get("boost_factor", boost_factor)
        focus_labels = retune_req.get("focus_labels", focus_labels)

    code_path = state.get("classifier_code_path") or os.path.join(OUTPUT_DIR, "classifier.py")
    os.makedirs(os.path.dirname(code_path) or ".", exist_ok=True)
```

Processing Agent

Classifier Agent

Evaluator Agent

Manager Agent

Explanation Agent

Evaluator Agent

- Validates predictions: required columns, labels, probabilities, confidence and split
- Scores the requested split: validation during retuning, test for the final report
- Computes overall accuracy + class accuracy for up/down/neutral
- Adds per-class support, so a missing class is not confused with model collapse

→ evaluation_report.json

```
def build_graph(checkpointer):  
    from langgraph.graph import StateGraph, START, END  
  
    builder = StateGraph(EvaluatorState)  
  
    builder.add_node("load_inputs", load_inputs)  
    builder.add_node("evaluate", evaluate)  
    builder.add_node("write_report", write_report)  
  
    builder.add_edge(START, "load_inputs")  
    builder.add_edge("load_inputs", "evaluate")  
    builder.add_edge("evaluate", "write_report")  
    builder.add_edge("write_report", END)  
  
    return builder.compile(checkpointer=checkpointer)
```

Processing Agent

Classifier Agent

Evaluator Agent

Manager Agent

Explanation Agent

Manager Agent

- The gate: compares accuracy to target and decides retune or proceed (pure rules)
- Won't let an "always-neutral" run fake a pass (class-collapse floor)
- If a retune makes things worse, reverts to the best settings and nudges gently (stateful loops)
- Finalizes on the best iteration, not the last one
- Stops on convergence or an iteration cap; samples ~300 rows for explanation

```
def build_graph(checkpointer):  
    from langgraph.graph import StateGraph, START, END  
  
    b = StateGraph(ManagerState)  
    b.add_node("decide", decide)  
    b.add_node("rationale", rationale)  
    b.add_node("write_retune", write_retune)  
    b.add_node("proceed", proceed)  
    b.add_node("finalize", finalize)  
    b.add_conditional_edges(START, route_entry, {"decide": "decide", "proceed": "proceed"})  
    b.add_edge("decide", "rationale") # gate first, then rationale  
    b.add_conditional_edges("rationale", route_after_decide, {"retune": "write_retune", "sample": "proceed"},  
                           )  
    b.add_edge("write_retune", END)  
    b.add_edge("proceed", END)  
    b.add_edge("finalize", END)  
    return b.compile(checkpointer=checkpointer)
```

Processing Agent

Classifier Agent

Evaluator Agent

Manager Agent

Explanation Agent

Explanation Agent

- Uses a local LLM (Ollama, llama3.2) to justify each prediction in one sentence
- Explains from the headline only — never sees the real outcome (no hindsight)
- Falls back to a safe placeholder if the LLM is unavailable — pipeline never crashes
- Feeds the qualitative evaluation (team scores explanations 1–5)
- → explanations.csv

```
EXPLANATION_SYSTEM_PROMPT = """You are a FINANCIAL ANALYST ...      # Persona
### CONTEXT ###    ... You do NOT know the actual outcome ...      # Sections + Option A
### THINKING ###   FIRST reason step by step, THEN answer          # Chain-of-Thought
### CONSTRAINTS ### Use ONLY the headline. DO NOT invent facts.    # CAPITALS
### OUTPUT ###     Return ONLY JSON: {"reasoning":..., "explanation":..} # Structured output
### EXAMPLE ###    Headline "... " -> { ... }                      # One-shot example
"""
```

Processing Agent

Classifier Agent

Evaluator Agent

Manager Agent

Explanation Agent

Results

Final accuracy*:

0.50

*accepting less coverage > better accuracy

Target accuracy:

0.60

(Baseline: 0.516)

Target set size:

2196

loop iterations:

5

Class accuracy:

up=0.36

down=0.05

neutral=0.76

Balanced accuracy (baseline: 0.33):

b.Accuracy: $(0.36 + 0.05 + 0.76) / 3$
= 0.39!

Manual evaluation average:

0.8

Interpretation

- The pipeline works end-to-end.
- The model does not meet the target.
- Performance then stalled, so the Manager Agent proceeded.
- The result suggests that headlines alone are a weak signal for next-day stock movement.

In the repo:
uv run marimo run dashboard.py
the presentation view

dashboard



A next-generation Python notebook

dashboard

A marimo notebook by jack from 0-to-ai-engineer

 marimo.io

Demo: Dashboard

Confidence Trade-off

The Model Knows When It's Clueless

Min. Confidence	Kept	Coverage	Accuracy
none	2,196	100%	0.497
≥ 0.45	171	7.8%	0.544
≥ 0.50	42	1.9%	0.738
≥ 0.60	24	1.1%	0.833

- Accuracy shoots up when we reject the low-confidence guesses
- 0.45 is the first time we beat the 0.516 baseline.
- But coverage plummets: less than 8% of headlines get a prediction.
- Above 0.50, the sample size shrinks to a teeny-tiny 42 rows

But it raises a question that for a model that you want to trust with financial decisions, could this be good?

Limitations & Next Steps

1. Original accuracy target was too optimistic

- 60% accuracy was our initial threshold.
- All-neutral baseline reaches 51.6%.
- A “correct” model must first beat that baseline.

2. Standard accuracy is misleading

- Standard accuracy can reward overpredicting neutral.
- Balanced accuracy evaluates up/down/neutral equally.
- Our balanced accuracy: 0.39 vs. 0.33 chance baseline.

3. Headline-only T+1 prediction is noisy

- One headline may not contain enough signal.
- Movement also depends on market, sector, volatility & expectations.
- So the task setup itself may need to change.

1. Update evaluation strategy

- Report both accuracy and balanced accuracy.
- Use the 51.6% all-neutral baseline as the minimum benchmark to beat.

2. Re-run with a revised target

- Replace the original 0.60 threshold with a baseline-aware threshold.
- A model should only be considered successful if it beats the majority-class baseline.#

3. Test alternative task setups

- Binary classification: up vs. down
- Fine-tuning in the loop
- Smaller or redefined neutral class
- Longer prediction horizon: T+3 or T+5 instead of T+1

Takeaway and what we learned

We successfully built a reproducible multi-agent NLP pipeline that processes financial headlines, predicts stock movement, evaluates performance, retunes, and generates explanations. The pipeline works end-to-end, but the final accuracy not going over the baseline and defaulting to neutral unless “sure” shows that headline-only next-day prediction is very difficult.

Do you have any questions?

Feel free to reach out!